Consultores y Modificadores

Dpto. Lenguajes y Sistemas Informáticos
Universidad de Granada

Programación y Diseño Orientado a Objetos

(Curso 2024-2025)

Créditos

- Las siguientes imágenes e ilustraciones son libres y se han obtenido de:
  - Emojis, `https://pixabay.com/images/id-2074153/`
- El resto de imágenes e ilustraciones son de creación propia, al igual que los ejemplos de código

Objetivos

- Saber crear y usar consultores y modificadores, tanto en Java como en Ruby
- Ser conscientes de la problemática de devolver o asignar referencias a objetos

Contenidos

1. Consultores
2. Modificadores
3. Ejemplos
   - Java
   - Ruby
4. Problemática de devolver (o asignar) referencias

Consultores

- Métodos encargados de devolver el valor de un atributo
- No tienen necesariamente que limitarse a devolver ese valor. Pueden devolverlo modificado, o una copia del mismo, etc.
- Pueden ser de clase o de instancia
- Habitualmente se nombran: `getAtributo()` en Java
- Habitualmente se nombran: `atributo` en Ruby
- Solo deben crearse los consultores que realmente sean necesarios
  - Se expone el estado interno al exterior
    - ★ ¿Se pueden usar dentro de los constructores?

Modificadores

- Métodos encargados de modificar el valor de un atributo
- No tienen necesariamente que limitarse a fijar ese valor.
  Pueden y deben controlar las restricciones sobre ese atributo
- Pueden ser de clase o de instancia
- Habitualmente se nombran: `setAtributo(...)` en Java
- Habitualmente se nombran: `atributo` en Ruby
- Solo deben crearse los modificadores que realmente sean necesarios
  - Se expone el estado interno al exterior
    - ★ ¿Se pueden usar dentro de los constructores?

Ejemplos

Java: Consultores y modificadores

```java
public class Persona {

  private static final int MAYORIAEDAD = 18; //Atributo de clase
  private LocalDateTime fechaNacimiento;     //Atributo de instancia

  Persona (LocalDateTime fecha) {
      fechaNacimiento = fecha;
  }

  public static int getMayoriaEdad() {
      return MAYORIAEDAD;
  }

  public LocalDateTime getFechaNacimiento() {
      // Se devuelve al exterior una referencia a la fecha de nacimiento
      // Podría ser modificada desde fuera
      return fechaNacimiento;
  }

  public void setFechaNacimiento(LocalDateTime fecha) {
      // Añadir comprobaciones relativas a las restricciones sobre la edad
      // Se está asignando una referencia a un objeto que ya está siendo referenciado
      //   desde fuera de la clase
      fechaNacimiento = fecha;
  }
}
```

Ejemplos

Java: Usando la clase anterior

```
1 Persona p=new Persona(LocalDateTime.of (2000,7,5,0,0));
2
3 // utilizamos el modificador
4 p.setFechaNacimiento(LocalDateTime.of (1950,7,5,0,0));
5
6 // utilizamos el consultor
7 System.out.println (p.getFechaNacimiento());
8
9 // utilizamos el consultor de clase
10 System.out.println (Persona.getMayoriaEdad());
```

Ejemplos

Ruby: Consultores y modificadores

```ruby
require 'date'

class Persona

  @@MAYORIA_EDAD = 18 # Atributo de clase

  def initialize (fecha)
    @fecha_nacimiento = fecha
  end

  attr_reader   :fecha_nacimiento   # consultor
  attr_writer   :fecha_nacimiento   # modificador
  attr_accessor :fecha_nacimiento   # consultor + modificador


  def self.MAYORIA_EDAD=e   # modificador de clase
    @@MAYORIA_EDAD = e
  end
end
```

Ejemplos

Ruby: Consultores y modificadores

```ruby
require 'date'

class Persona

  @@MAYORIA_EDAD = 18 # Atributo de clase

  def initialize (fecha)
    @fecha_nacimiento = fecha
  end


  def fecha_nacimiento     # consultor
    # Se devuelve al exterior del objeto una referencia a la fecha
    @fecha_nacimiento
  end

  def fecha_nacimiento=fecha    # modificador
    # ¿ Restricciones ?
    # Se asigna una referencia a un objeto que ya es referenciado desde fuera
    @fecha_nacimiento = fecha
  end

  def self.MAYORIA_EDAD=e # modificador de clase
    @@MAYORIA_EDAD = e
  end
end
```

Ejemplos

Ruby: Usando la clase anterior

```ruby
p=Persona.new(Date.new(2000,7,3))

# utilizamos el modificador
p.fecha_nacimiento=Date.new(2000,8,3)

# utilizamos el consultor
puts p.fecha_nacimiento

# utilizamos el modificador de clase
Persona.MAYORIA_EDAD=21
```

Ejemplos

Ruby: Consultores y modificadores implícitos

```ruby
class UnaClase

  attr_reader   :atr1
  attr_accessor :atr2
  attr_writer   :atr3

  def initialize (un, dos, tres)
    @atr1 = un
    @atr2 = dos
    @atr3 = tres
  end

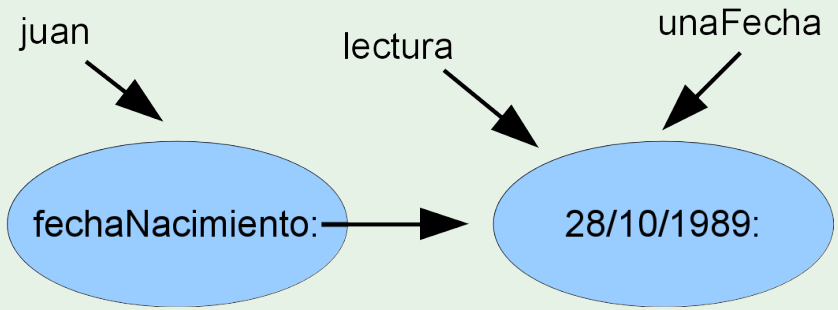
end

obj = UnaClase.new(1,2,3)
obj.atr2 = 8
puts obj.inspect
obj.atr2 = 9
puts obj.inspect
obj.atr3 = 7
puts obj.inspect
puts obj.atr1
puts obj.atr2
#puts obj.atr3 # no existe consultor
#obj.atr1 = 23 # no existe modificador
```

Problemática de devolver (o asignar) referencias

Java: Asignación y devolución de referencias

```
1  class Persona {
2    private GregorianCalendar fechaNacimiento;
3
4    Persona (GregorianCalendar nace) {
5      fechaNacimiento = nace;
6    }
7
8    GregorianCalendar getFechaNacimiento () {
9      return fechaNacimiento;
10   }
11
12   // . . .
13 }
14
15 GregorianCalendar unaFecha = new GregorianCalendar (1989,10,28);
16
17 Persona juan = new Persona (unaFecha);
18 System.out.println(juan.toString());    // Nací el 28/10/1989
19
20 GregorianCalendar lectura = juan.getFechaNacimiento();
21 lectura.set (1985,5,13);
22 System.out.println(juan.toString());    // Nací el 13/5/1985
23 unaFecha.set (2001,1,1);
24 System.out.println(juan.toString());    // Nací el 1/1/2001
```



★ ¿Soluciones?

Consultores y Modificadores → *Diseño* ←

- Crear solo los que sean realmente necesarios
- Tener en cuenta si se devuelven (o se asignan) referencias
  - *En lenguajes como Java o Ruby todas las variables son referencias (punteros)*
    - ★ Salvo los tipos primitivos: int, float, ...
      Los String tampoco deben preocuparnos



- No hay una regla a aplicar en todos los casos
  - A veces interesa devolver (o asignar) una referencia
  - Otras veces interesa devolver (o asignar) una copia
  - Puede depender de *a quién* se le dé (o *de dónde*) venga
- Hay que decidirlo evaluando cada situación

Java: Asignación de referencias

```
1 objeto = new Clase();
2 otroObjeto = objeto;
3 // Un ÚNICO objeto con dos referencias
```

Ruby: Asignación de referencias

```
1 objeto = Clase.new;
2 otroObjeto = objeto;
3 # Un ÚNICO objeto con dos referencias
```

Consultores y Modificadores

Dpto. Lenguajes y Sistemas Informáticos
Universidad de Granada

Programación y Diseño Orientado a Objetos

(Curso 2024-2025)